

Contents

1	Math	1
1.1	Fpow	1
1.2	Combination	1
1.3	Extgcd	1
2	Shortest Path	1
2.1	Dijkstra	1
2.2	Bellman-Ford	1
2.3	Floyd-Warshall	1
3	DP	1
3.1	0-1 Knapsack	1
3.2	Unbounded Knapsack	2
3.3	SOS DP	2
3.4	O(lgn) LIS	2
4	Graph	2
4.1	Euler Path	2
5	Data Structure	2
5.1	Segment Tree	2
5.2	Binary Indexed Tree	2
6	Others	3
6.1	Discretization	3
7	Advanced Dynamic Programming	3
7.1	Bitmask DP	3
7.2	Sum Over Subsets (SOS DP) & 高維前綴和	3
7.3	Counting DP	3
7.4	Probabilistic DP	4

1 Math

1.1 Fpow

```
int fpow(int x, int n, int mod) {
    int res = 1;
    while (n) {
        if (n & 1) res = res * x % mod;
        x = x * x % mod;
        n >>= 1;
    }
    return res;
}
```

1.2 Combination

```
#define MXN 1'000'005
#define N 1'000'000
#define ll long long

ll fac[MXN], inv[MXN];
fac[0] = 1; // 0! = 1
for(int i = 1; i <= N; i++) fac[i] = fac[i-1] * i % mod;
inv[N] = fpow(fac[N], mod-2); // 快速幂
for(int i = N-1; i >= 0; i--) inv[i] = inv[i+1] * (i+1) % mod;

ll C(int n, int k) {
    return fac[n] * inv[k] % mod * inv[n-k] % mod;
}
```

1.3 Extgcd

```
#define ll long long
int extgcd(int a, int b, int &x, int &y) {
    if(b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int GCD = extgcd(b, a % b, y, x);
    y -= a / b * x;
    return GCD;
}
```

```
int main() {
    int a, b, x, y;
    cin >> a >> b;
    int GCD = extgcd(a, b, x, y);
}
```

2 Shortest Path

2.1 Dijkstra

```
#define pii pair<int,int>
#define INF 1e18
vector<pair<int,int>>vec[N];
void dijkstra(int s, int t) { // 起點 · 終點
    int dis[N];
    for(int i=0; i<N; i++) {
        dis[i] = INF; // 值要設為比可能的最短路徑權重還要大的值
    }
    dis[s] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({dis[s], s});
    while(pq.empty() == 0) {
        int u = pq.top().second;
        pq.pop();
        if(vis[u]) continue;
        vis[u] = 1;
        for(auto [v, w] : vec[u]) {
            if(dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }
}
```

2.2 Bellman-Ford

```
struct edge {
    int v, w;
};
int s, n; // 起點 點數量
vector<edge> E[MXN];
vector<int> dis(MXN, INF), cnt(MXN), inq(MXN);
// cnt記錄最短路經過幾條邊
// inq記錄節點是否在queue裡面
queue<int> Q;
dis[s] = 0; Q.push(s); inq[s] = 1;
while(Q.size()) {
    int u = Q.front(); Q.pop();
    inq[u] = 0;
    for(auto [v, w] : E[u]) {
        if(dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            cnt[v] = cnt[u] + 1;
            if(cnt[v] >= n) {有負環}
            if(!inq[v]) Q.push(v), inq[v] = 1;
        }
    }
}
```

2.3 Floyd-Warshall

```
for(int k=1; k<=N; k++) { // 窮舉中繼點k
    for(int i=1; i<=N; i++) {
        for(int j=1; j<=N; j++) { // 窮舉點對(i,j)
            dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
        }
    }
}
```

3 DP

3.1 0-1 Knapsack

```

for(int i = 1; i <= n; ++i){
    for(int w = W; w >= 0; --w)
        dp[w] = max(w - c[i] >= 0 ? dp[w - c[i]] + v[i] : 0, dp[w]);

    // Backtracing
    for(int i = 1; i <= n; ++i){
        for(int w = 0; w <= W; ++w) {
            //dp[i][w] 考慮前 i 個物品且負重上限為 w 的答案。
            int a = w - c[i] >= 0 ? dp[i - 1][w - c[i]] + v[i] : 0;
            int b = dp[i - 1][w];
            if(a > b){
                dp[i][w] = a;
                R[i][w] = w - c[i];
            } else {
                dp[i][w] = b;
                R[i][w] = w;
            }
        }
    }

    int i = n, w = W;
    vector<int> ans;
    while(i){
        if(w != R[i][w]) ans.push_back(i); //不一樣代表有取
        w = R[i][w];
        --i;
    }
}

```

3.2 Unbounded Knapsack

```

for(int w = 0; w <= W; ++w){
    for(int i = 1; i <= n; ++i){
        dp[w] = max(dp[w], w - c[i] >= 0 ? dp[w - c[i]] + v[i] : 0);
    }
}

```

3.3 SOS DP

```

for(int i = 0; i < (1 << N); ++i)
    F[i] = A[i];
for(int i = 0; i < N; ++i)
    for(int mask = 0; mask < (1 << N); ++mask)
        if(mask & (1 << i))
            F[mask] += F[mask ^ (1 << i)];
//窮舉子集合
for(int s = mask; s; s = (s - 1) & mask; )

```

3.4 O(lgn) LIS

```

#define all(v) (v).begin(), (v).end()

vector<int> dp;
for(int i = 0; i < n; ++i){
    auto it = lower_bound(all(dp), A[i]);
    if(it == dp.end()){
        dp.push_back(A[i]);
    } else {
        *it = A[i];
    }
}
cout << dp.size() << '\n';

```

4 Graph

4.1 Euler Path

```

vector<int> path;
void dfs(int x){
    while(!edge[x].empty()){
        int u = edge[x].back();
        edge[x].pop_back();
        dfs(u);
    }
    path.push_back(x);
}

int main(){
    dfs(st);
    reverse(path.begin(), path.end());
    for(int i: path) cout << i << ' ';
}

```

5 Data Structure

5.1 Segment Tree

```

#define cl(x) (x << 1)
#define cr(x) (x << 1) + 1
const int N;
int seg[4 * N];
int arr[N];
void pull(int id){
    seg[id] = max(seg[cl(id)], seg[cr(id)]);
}

void build(int id, int l, int r){
    if(l == r){
        seg[id] = arr[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(cl(id), l, mid);
    build(cr(id), mid + 1, r);
    pull(id);
}

void update(int id, int l, int r, int x, int v){
    if(l == r){
        seg[id] = v;
        return;
    }
    int mid = (l + r) >> 1;
    if(x <= mid){
        update(cl(id), l, mid, x, v);
    }
    if(mid < x){
        update(cr(id), mid + 1, r, x, v);
    }
    pull(id);
}

int query(int id, int l, int r, int sl, int sr){
    if(sl <= l && r <= sr){ //目前這個區間在查詢區間內
        return seg[id];
    }
    int mid = (l + r) >> 1, res = 0;
    if(sl <= mid){ //左區間跟查詢區間有交集
        res = max(res, query(cl(id), l, mid, sl, sr));
    }
    if(mid < sr){ //右區間跟查詢區間有交集
        res = max(res, query(cr(id), mid + 1, r, sl, sr));
    }
    return res;
}

```

5.2 Binary Indexed Tree

```

struct Binary_Indexed_Tree{
    int n;
    vector<long long> bit;
    int lowbit(int x){
        return x & -x;
    }
    void init(int _n){
        n = _n + 1;
        bit = vector<long long>(n, 0);
    }
}

```

```

    }
    void update(int x, int v) {
        for (; x < n; x += lowbit(x)) {
            bit[x] += v;
        }
    }
    long long query(int x) {
        long long ret = 0;
        for (; x > 0; x -= lowbit(x)) {
            ret += bit[x];
        }
        return ret;
    }
} BIT;

```

6 Others

6.1 Discretization

```

//vector<int> arr;
vector<int> tmp(arr); //將arr複製到tmp
sort(tmp.begin(), tmp.end());
tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
for (int i = 0; i < n; i++)
    arr[i] = lower_bound(tmp.begin(), tmp.end(), arr[i]) - tmp.begin();

```

7 Advanced Dynamic Programming

7.1 Bitmask DP

位元壓縮 DP，泛指狀態是由二進制表示的 DP。通常可以表示集合，0 表示不取，1 表示取。假設有 n 個東西要考慮，使用長度為 n 的 `bitstring` 表示狀態，複雜度通常為 $O(2^n)$ 。看到題目範圍 $n \leq 20$ 即可考慮此解法。

搭電梯問題 (Elevator Problem)
 n 個人排隊，每個人重 w_i ，要搭一台負重 x 的電梯，問最少要搭幾趟？利用 $O(2^n)$ 窮舉最後上電梯的人，減少了窮舉 $n!$ 排列的時間複雜度。

```

#define pp pair<int, int>
#define ff first
#define ss second

pp dp[(1<<20)]; // State up to 2^20

void solve() {
    int n, x;
    cin >> n >> x;
    vector<int> w(n);
    for (auto &c : w) cin >> c;

    // Initialize: maximum n rides, so set n+1 as infinity
    for (int s = 0; s < (1<<n); ++s) dp[s].ff = n + 1;

    // Empty set weight is x (forces first item to start a new ride)
    dp[0].ss = x, dp[0].ff = 0;

    // Iterate through all state sets
    for (int s = 1; s < (1<<n); ++s) {
        for (int i = 0; i < n; ++i) {
            // Check if person i is in the current set
            if (!(s & (1<<i))) continue;

            // ... Transition logic goes here ...
        }
    }
}

```

Hamiltonian Path / TSP

圖中有 n 個點，詢問是否能走過所有點且每個點僅經過一次 ($O(n \cdot 2^n)$)。需要記錄經過了哪些點，以及停留在哪個點。

```

// Initialize: each node can reach itself
for (int i = 0; i < n; ++i) {
    dp[(1<<i)][i] = true;
}

```

```

// Iterate through all subsets
for (int s = 1; s < (1<<n); ++s) {
    for (int j = 0; j < n; ++j) {
        // Continue if vertex j is not in state s
        if (s & (1<<j)) continue;

        // Iterate all endpoints in state s
        for (int i = 0; i < n; ++i) {
            // s must pass through i
            if (!(s & (1<<i))) continue;

            // If there is a directed edge from i to j
            if (edge[i][j]) {
                dp[s | (1<<j)][j] |= dp[s][i];
            }
        }
    }
}

bool ans = false;
for (int i = 0; i < n; ++i) ans |= dp[(1<<n)-1][i];

```

7.2 Sum Over Subsets (SOS DP) & 高維前綴和

目標是解決求給定集合的所有子集和：

$$F(S) = \sum_{x \subseteq S} A(x)$$

直接窮舉子集複雜度為 $O(3^n)$ ，使用 SOS DP 則可優化至 $O(n \cdot 2^n)$ 。

高維前綴和概念 (High-Dimensional Prefix Sum)

SOS DP 的本質其實就是高維度的前綴和推廣：

- 一維前綴和：沿著單一方向累加， $F[x] += F[x-1]$ 。
- 二維前綴和：先固定 Y 對 X 做一維前綴和 ($F[x][y] += F[x-1][y]$)，再固定 X 對 Y 做一維前綴和 ($F[x][y] += F[x][y-1]$)。幾何上就是把一條條的前綴和 (線) 疊加成一個矩形 (面)。
- N 維前綴和 (SOS DP)**：在 `bitmask` 中，共有 n 個維度，每個維度的大小只有 0 和 1。我們只需依序窮舉每一維 i ，若當前狀態 s 在該維度是 1 (即 $s \& (1 \ll i)$ 為真)，就將長度較短的狀態 (即該維度為 0 的狀態 $s \oplus (1 \ll i)$) 累加到 $F[s]$ 中。

```

// Precompute state weights (single point initialization)
for (int s = 0; s < (1<<n); ++s) {
    F[s] = A[s];
}

// Iterate through each dimension (bit)
for (int i = 0; i < n; ++i) {
    // Iterate through all states (points in the n-dimensional space)
    for (int s = 0; s < (1<<n); ++s) {
        // If the i-th bit of state s is 1 (meaning it includes this dimension)
        if (s & (1 << i)) {
            // Accumulate subset sums: F[s] += F[s - 1] on the i-th dimension
            F[s] += F[s ^ (1 << i)];
        }
    }
}

```

7.3 Counting DP

用於計算方法數的 DP，通常可以配合排列組合與排容原理。

障礙物棋盤走路計數問題

給定 $n \times m$ 棋盤與 N 個障礙物，只能向下或右走，求起點到終點且不經過障礙物的方法數。利用總和扣除不合法路徑的方法：

$$dp[D] = cnt(O, D) - \sum_{\{k\}} cnt(k, D) \cdot dp[k]$$

轉移順序：將點先按 X 排序，再按 Y 排序，確保 `partial-ordering` 正確。

```

vector<pair<int, int>> A(N);
sort(A.begin(), A.end());

// Insert start point at index 0 and end point at back
A.insert(A.begin(), pair<int, int>{1, 1});
A.push_back(pair<int, int>{n, m});

vector<int> dp(N + 5);

for(int i = 1; i <= N + 1; ++i) {
    // Calculate total valid permutations without
    // obstacles
    dp[i] = cnt(0, i);
    auto [x, y] = A[i];

    // Subtract combinations that pass through other
    // obstacles
    for(int j = 1; j < i; ++j) {
        auto [xx, yy] = A[j];

        // Ensure obstacle j is strictly within the
        // path to obstacle i
        if(xx <= x && yy <= y) {
            dp[i] -= cnt(j, i) * dp[j];
        }
    }
}

cout << dp[N+1];
}

auto &cur = dp[i][j];

// Case 1: Princess catches a white mouse
// immediately
cur += (1d)i / (i + j);

// Probability that both princess and dragon
// catch a black mouse
1d x = (1d)j / (i + j) * (j - 1) / (i + j - 1);

// Case 2: A black mouse escapes after they
// both catch black mice
if (j >= 3) {
    cur += x * (j - 2) / (i + j - 2) * dp[i][j
-3];
}

// Case 3: A white mouse escapes after they
// both catch black mice
if (j >= 2) {
    cur += x * i / (i + j - 2) * dp[i-1][j-2];
}
}
}

```

7.4 Probabilistic DP

將機率丟到 DP 上做轉移，或者計算期望值（利用期望值的線性性質 $E[X + c \cdot Y] = E[X] + c \cdot E[Y]$ ）。

骰子問題 (Dice Probabilities)

擲 n 次六面骰子，所有結果相加落在 $[a, b]$ 的機率。

```

// dp[i][j] represents the probability of reaching sum
// j after i rolls
double dp[N+1][6*N+1];

// Initialize the first roll probability
for(int i = 1; i <= 6; ++i) {
    dp[1][i] = 1.0 / 6.0;
}

for(int i = 2; i <= n; ++i) {
    // Max sum after n rolls is 6*n (assuming n <= 100,
    // max 600)
    for(int w = 1; w <= 600; ++w) {
        // Iterate through all 6 dice faces
        for(int j = 1; j <= 6; ++j) {
            if(w >= j) {
                dp[i][w] += dp[i-1][w-j];
            }
        }
        // Divide once per state to prevent precision
        // errors
        dp[i][w] /= 6.0;
    }
}

double ans = 0.0;
for(int i = a; i <= b; ++i) {
    ans += dp[n][i];
}
}

```

抓老鼠期望值問題 (Mice Probability)

袋中有 w 隻白鼠、 b 隻黑鼠。雙方輪流抓，抓到白鼠贏，每次抓完黑鼠有機率跑出一隻。

```

using ld = long double;
ld dp[1005][1005];

// Initialize: when only white mice remain, princess
// wins 100%
for (int i = 1; i <= w; i++) dp[i][0] = 1;
// Initialize: when only black mice remain, princess
// wins 0%
for (int i = 1; i <= b; i++) dp[0][i] = 0;

for (int i = 1; i <= w; i++) {
    for (int j = 1; j <= b; j++) {

```